

Week6_예습과제_박연정

≡ 범위 5장

3. 회귀

01. 회귀 소개

데이터 값이 평균과 같은 일정한 값으로 돌아가려는 경향을 이용한 통계학 기법

회귀: 여러 개의 독립변수와 한 개의 종속변수 간의 상관관계를 모델링하는 기법을 통칭

예시: 아파트의 방 개수, 방 크기, 주변 학군 등 여러 개의 독립변수에 따라 아파트의 가격이라는 종속변수가 어떤 관계를 갖는지 모델링하고 예측

머신러닝 관점에서

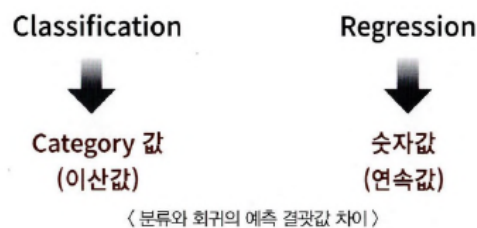
- 독립변수는 피쳐
- 종속변수는 결정 값

주어진 피쳐와 결정 값 데이터 기반에서 학습을 통해 최적의 회귀 계수를 찾아내는 것이 핵심

독립변수 개수	회귀 계수의 결합
1개: 단일 회귀	선형: 선형 회귀
여러 개: 다중 회귀	비선형: 비선형 회귀

〈 회귀 유형 구분 〉

지도학습은 분류와 회귀로 나뉘는데, 두 기법의 가장 큰 차이는 분류는 예측값이 이산값이고 회귀는 연속값이라는 점

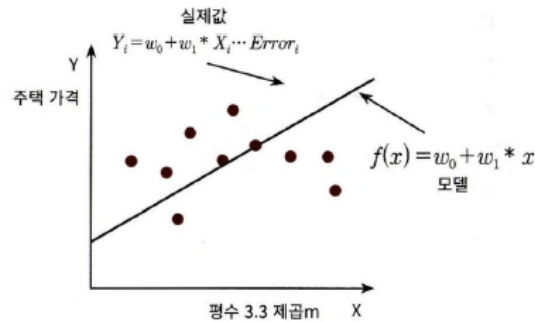


선형 회귀: 실제 값과 예측값 차이(오류의 제곱 값)를 최소화하는 직선형 회귀선을 최적화하는 방식

- 일반 선형 회귀는 규제를 적용하지 않은 모델
- 규제 방법에 따라 다시 별도의 유형으로 나눌 수 있음
- 규제: 일반적인 선형 회귀의 과적합 문제를 해결하기 위해 회귀 계수에 패널티 값을 적용하는 것

02. 단순 선형 회귀를 통한 회귀 이해

단순 선형 회귀: 독립변수, 종속변수 모두 하나인 선형 회귀



잔차: 실제 값과 회귀 모델의 차이에 따른 오류 값

- 오류 값은 +, - 가 될 수 있으므로 절대값을 취해 더하거나 제곱을 구해서 더하는 방식(RSS)을 취함
- 일반적으로 미분 등의 계산을 편리하게 하기 위해 제곱 방식으로 오류 합을 구함
 - RSS는 w 변수(회귀계수)가 중심 변수임
 - RSS에서 학습데이터로 입력되는 독립변수, 종속변수는 모두 상수로 간주함

$$RSS(w_0, w_1) = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 * x_i))^2$$

(i는 1부터 학습 데이터의 총 건수 N까지)

- 회귀에서 RSS는 비용(Cost)이며 w 변수로 구성되는 RSS를 비용함수(손실함수)라고 함.

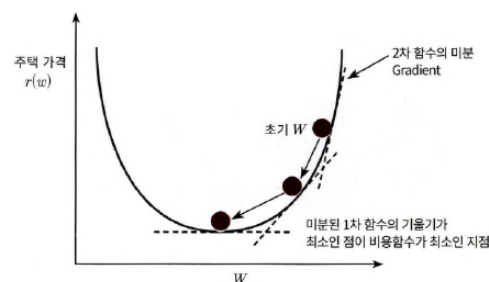
최적의 회귀 모델을 만든다는 것은

- 전체 데이터의 잔차(오류값) 합이 최소가 되는 모델을 만든다는 의미
- 동시에 오류 값의 합이 최소가 될 수 있는 최적의 회귀계수를 찾는다는 의미

03. 비용 최소화하기- 경사 하강법(Gradient Descent) 소개

점진적으로 반복적인 계산을 통해 W 파라미터 값을 업데이트하면서 오류 값이 최소가 되는 W 파라미터를 구하는 방식

“어떻게 하면 오류가 작아지는 방향으로 W 값을 보정할 수 있을까?”



$$\frac{\partial R(w)}{\partial w_1} = \frac{2}{N} \sum_{i=1}^N -x_i * (y_i - (w_0 + w_1 x_i)) = -\frac{2}{N} \sum_{i=1}^N x_i * (\text{실제값}_i - \text{예측값}_i)$$

$$\frac{\partial R(w)}{\partial w_0} = \frac{2}{N} \sum_{i=1}^N -(y_i - (w_0 + w_1 x_i)) = -\frac{2}{N} \sum_{i=1}^N (\text{실제값}_i - \text{예측값}_i)$$

04. 사이킷런 LinearRegression을 이용한 보스턴 주택 가격 예측

LinearRegression 클래스 - Ordinary Least Squares

예측값과 실제 값의 RSS를 최소화해 OLS 추정 방식으로 구현한 클래스

LinearRegression 클래스는 fit() 메서드로 X, y 배열을 입력받으면 회귀 계수인 W를 coef_ 속성에 저장함

- 다중공산성 문제: 피쳐 간의 상관관계가 매우 높은 경우 분산이 매우 커져 오류에 민감해짐

회귀 평가 지표

평가 지표	설명	수식
MAE	Mean Absolute Error(MAE)이며 실제 값과 예측값의 차이를 절댓값으로 변형해 평균한 것입니다.	$MAE = \frac{1}{n} \sum_{i=1}^n Y_i - \hat{Y}_i $
MSE	Mean Squared Error(MSE)이며 실제 값과 예측값의 차이를 제곱해 평균한 것입니다.	$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$
RMSE	MSE 같은 오류의 제곱을 구함으로 실제 오류 평균보다 더 커지는 특성이 있으므로 MSE에 루트를 씌운 것이 RMSE(Root Mean Squared Error)입니다.	$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2}$
R ²	분산 기반으로 예측 성능을 평가합니다. 실제 값의 분산 대비 예측값의 분산 비율을 지표로 하며, 1에 가까울수록 예측 정확도가 높습니다.	$R^2 = \frac{\text{예측값 Variance}}{\text{실제값 Variance}}$

평가 방법	사이킷런 평가 지표 API	Scoring 함수 적용 값
MAE	metrics.mean_absolute_error	'neg_mean_absolute_error'
MSE	metrics.mean_squared_error	'neg_mean_squared_error'
RMSE	metrics.mean_squared_error를 그대로 사용하되 squared 파라미터를 False로 설정	'neg_root_mean_squared_error'
MSLE	metrics.mean_squared_log_error	'neg_mean_squared_log_error'
R ²	metrics.r2_score	'r2'

과거 버전의 사이킷런은 RMSE를 계산하는 함수가 제공되지 않았지만, 0.22 버전부터는 RMSE를 위한 함수를 제공합니다. RMSE를 구하기 위해서는 MSE를 위한 metrics.mean_squared_error() 함수를 그대로 사용하되, squared 파라미터를 False로 지정해 사용합니다. mean_squared_error() 함수는 squared 파라미터가 기본적으로 True 입니다. 즉 MSE는 사이킷런에서 mean_squared_error(실제값, 예측값, squared=True)이며 RMSE는 mean_squared_error(실제값, 예측값, squared=False)를 이용해서 구합니다.

LinearRegression을 이용해 보스턴 주택 가격 회귀 구현



05. 다항 회귀와 과(대)적합/과소적합 이해

다항 회귀 이해

다항 회귀도 선형 회귀

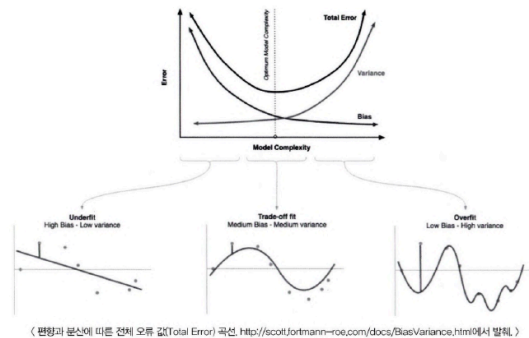
비선형 함수를 선형 모델에 적용시키는 방법을 통해 구현

- PolynomialFeatures 클래스를 통해 피쳐를 Polynomial(다항식) 피쳐로 변환

- degree 파라미터를 통해 입력받은 단항식 피처를 degree에 해당하는 다항식 피처로 변환
- fit(), transform() 메서드를 통해 변환 작업 수행

다항 회귀를 이용한 과소적합 및 과적합 이해

편향-분산 트레이드오프(Bias-Variance Trade off)



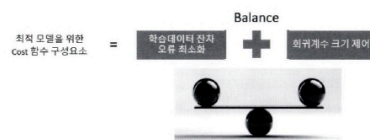
높은 편향/낮은 분산에서 과소적합되기 쉽고

낮은 편향/높은 분산에서 과적합되기 쉬움

편향과 분산이 서로 트레이드오프를 이루면서 Cost 값이 최대로 낮아지는 모델을 구축하는 것이 가장 효율적인 머신러닝 예측 모델을 만드는 방법

06. 규제 선형 모델- 릿지, 라쏘, 엘라스틱넷

규제 선형 모델의 개요



이렇게 회귀 계수의 크기를 제어해 과적합을 개선하려면 비용(Cost) 함수의 목표가 다음과 같이 $RSS(W) + \alpha * \|W\|^2$ 를 최소화하는 것으로 변경될 수 있습니다.

$$\text{비용 함수 목표} = \text{Min}(RSS(W) + \alpha * \|W\|^2)$$

alpha: 학습 데이터 적합 정도와 회귀 계수 값의 크기 제어를 수행하는 튜닝 파라미터

- alpha 값을 크게 하면 비용 함수는 회귀 계수 W의 값을 작게 해 과적합을 개선
- alpha 값을 작게 하면 회귀 계수 W 값이 커져도 어느 정도 상쇄 가능하므로 학습데이터 적합을 더 개선

L2 규제: W 제공에 대해 패널티를 부여

L1 규제: W의 절댓값에 대해 패널티 부여

릿지 회귀

릿지(Ridge): 선형 회귀에 L2 규제를 추가한 회귀 모델

- L2 규제는 상대적으로 큰 회귀 계수값의 예측 영향도를 감소시키기 위해서 회귀 계수값을 더 작게 만드는 규제 모델

사이킷런은 **Ridge 클래스**를 통해 릿지 회귀를 구현

라쏘 회귀

라쏘(Lasso): 선형 회귀에 L1 규제를 적용한 모델

- L1 규제는 예측 영향력이 작은 피처의 회귀 계수를 0으로 만들어 회귀 예측 시 피처가 선택되지 않게 함.
- L1 규제는 피처 선택 기능으로도 불림

사이킷런은 **Lasso 클래스**를 통해 라쏘 회귀 구현

엘라스틱넷 회귀

엘라스틱넷(ElasticNet): L2, L1 규제를 함께 결합한 모델

- 주로 피처가 많은 데이터 세트에 적용
- L1 규제로 피처의 개수를 줄임과 동시에
- L2 규제로 계수 값의 크기를 조정함

사이킷런은 **ElasticNet 클래스**를 통해서 엘라스틱넷 회귀 구현

선형 회귀 모델을 위한 데이터 변환

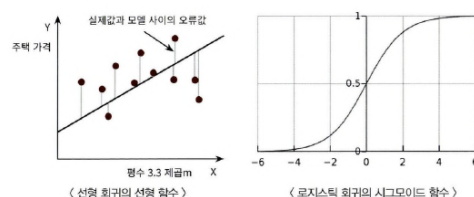
1. StandardScaler 클래스를 이용해 평균이 0, 분산이 1인 표준 정규 분포를 가진 데이터 세트로 변환하거나 MinMaxScaler 클래스를 이용해 최소값이 0이고 최대값이 1인 값으로 정규화를 수행합니다.
2. 스케일링/정규화를 수행한 데이터 세트에 다시 다항 특성을 적용하여 변환하는 방법입니다. 보통 1번 방법을 통해 예측 성능에 향상이 없을 경우 이와 같은 방법을 적용합니다.
3. 원래 값에 log 함수를 적용하면 보다 정규 분포에 가까운 형태로 값이 분포됩니다. 이러한 변환을 로그 변환(Log Transformation)이라고 부릅니다. 로그 변환은 매우 유용한 변환이며, 실제로 선형 회귀에서는 앞에서 소개한 1, 2 번 방법보다 로그 변환이 훨씬 많이 사용되는 변환 방법입니다. 왜냐하면 1번 방법의 경우 예측 성능 향상을 크게 기대하기 어려울 경우가 많으며 2번 방법의 경우 피처의 개수가 매우 많은 경우에는 다항 변환으로 생성되는 피처의 개수가 기하급수로 늘어나서 과적합의 이슈가 발생할 수 있기 때문입니다.

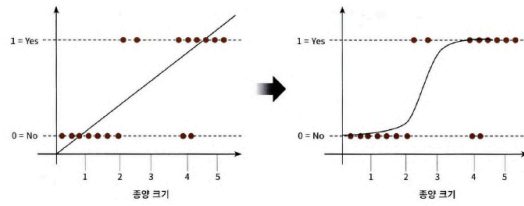
07. 로지스틱 회귀

로지스틱 회귀(Logistic Regression): 사실은 분류에 사용되는 선형 모델

- 매우 강력한 분류 알고리즘
- 일반적으로 이진 분류뿐 아니라
- 희소 영역의 분류(텍스트 분류) 영역에서 뛰어난 예측 성능을 보임

선형 함수의 회귀 최적선을 찾는 것이 아니라 시그모이드(Sigmoid) 함수 최적선을 찾고 이 시그모이드 함수의 반환 값을 확률로 간주해 확률에 따라 분류 결정



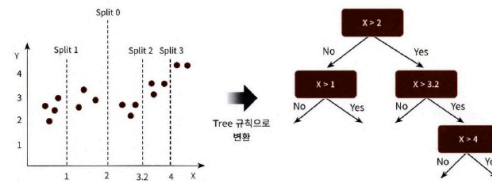


사이킷런은 로지스틱 회귀를 위해서 LogisticRegression 클래스를 제공합니다. LogisticRegression 클래스의 회귀 계수 최적화는 본 장의 초반부에 소개해 드린 경사 하강법 외에 다양한 최적화 방안을 선택할 수 있습니다. LogisticRegression 클래스에서 solver 파라미터의 'lbfgs', 'liblinear', 'newton-cg', 'sag', 'saga' 값을 적용해서 최적화를 선택할 수 있습니다.

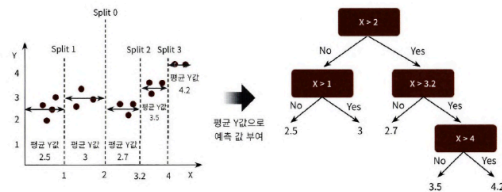
- lbfgs: 사이킷런 버전 0.22부터 solver의 기본 설정값입니다. 메모리 공간을 절약할 수 있고, CPU 코어 수가 많다면 최적화를 병렬로 수행할 수 있습니다.
- liblinear: 사이킷런 버전 0.21까지에서 solver의 기본 설정값입니다. 다차원이고 작은 데이터 세트에서 효과적으로 동작하지만 국소 최적해(Local Minimum)에 이수가 있고, 병렬로 최적화할 수 없습니다.
- newton-cg: 좀 더 정확한 최적화를 가능하게 하지만, 대용량의 데이터에서 속도가 많이 느려집니다.
- sag: Stochastic Average Gradient로서 경사 하강법 기반의 최적화를 적용합니다. 대용량의 데이터에서 빠르게 최적화합니다.
- saga: sag와 유사한 최적화 방식이며 L1 정규화를 가능하게 해줍니다.

08. 회귀 트리

이 데이터 세트의 X 피치를 결정 트리 기반으로 분할하면 X값의 균일도를 반영한 지니 계수에 따라 다음 그림의 왼쪽과 같이 분할할 수 있습니다. 루트 노드를 Split 0 기준으로 분할하고 이렇게 분할된 규칙 노드에서 다시 Split 1과 Split 2 규칙 노드로 분할할 수 있습니다. 그리고 Split 2는 다시 재귀적으로 Split 3 규칙 노드로 다음 그림의 오른쪽과 같이 트리 규칙으로 변환될 수 있습니다.



리프 노드 생성 기준에 부합하는 트리 분할이 완료됐다면 리프 노드에 소속된 데이터 값의 평균값을 구해서 최종적으로 리프 노드에 결정 값으로 할당합니다.



알고리즘	회귀 Estimator 클래스	분류 Estimator 클래스
Decision Tree	DecisionTreeRegressor	DecisionTreeClassifier
Gradient Boosting	GradientBoostingRegressor	GradientBoostingClassifier
XGBoost	XGBRegressor	XGBClassifier
LightGBM	LightGBMRegressor	LightGBMClassifier